Machine Learning Based Analog Circuit Fault Diagnosis
PDF 17 — Chapter 24: day12b_generate_noise_samples.py
Why Realistic Measurement Noise Was Added | Complete Gaussian Noise Mathematics

PART 1 — Why the Noise Generator Exists

Chapter 24 — Complete Python Source Code Explained Part 1 — day12b_generate_noise_samples.py (Based on your actual source code)

Before Starting

Your dataset was already complete. It contained Normal Open Short Drift faults. Question. Why create another fault? The answer is Reality.

Why This File Exists

LTspice is an ideal simulator. Question. Does real hardware produce perfect waveforms? No. Real circuits always contain noise. Examples Thermal Noise Johnson Noise Flicker Noise EMI Sensor Noise ADC Quantization Noise If our ML model is trained only on ideal signals, it may fail when deployed in the real world. That is why this script exists. Goal of This Script Professor asks What is the purpose of day12b_generate_noise_samples.py?

Perfect Answer

This script generates realistic noisy versions of the normal circuit responses by adding Gaussian noise with a fixed Signal-to-Noise Ratio (SNR). These noisy samples are added as a new fault class, making the dataset more representative of practical operating conditions.

Complete Pipeline

Select Normal Samples

Generate Gaussian Noise

Add Noise

Create Noise Class

Append Dataset

Save Updated CSV Import Section Your code imports

```
import os
```

```
import numpy as np
```

```
import pandas as pd
```

No new libraries. Everything uses NumPy and Pandas. Configuration Your code contains BASE Simply the project folder. Next CSV_IN This points to master_dataset_sweep.csv Question. Why? Because this script modifies the existing dataset. It doesn't create a new one from scratch. SNR One of the most important variables. SNR_DB = 25 Question. What is SNR? SNR means Signal-to-Noise Ratio. Formula

Signal Power

Noise Power

Usually measured in decibels. What Does 25 dB Mean? Question. Is 25 dB high noise? No. It means the signal is much stronger than the noise. Example Signal ▮▮▮▮▮▮▮▮▮▮ Noise ▮▮ Waveform is still recognizable. Exactly what happens in real circuits.

PROFESSOR ASKS

Why did you choose 25 dB?

EXCELLENT ANSWER

25 dB provides a moderate amount of measurement noise. The waveform remains physically meaningful while introducing realistic disturbances similar to those encountered in practical electronic systems.

Random Seed

Your code contains SEED = 42 Question. Why? Because random numbers normally change every run. Setting a seed makes the experiment reproducible. Example Without seed

Run 1

Different noise

Run 2

Different noise With seed

Run 1

Same noise

Run 2

Same noise Perfect for research.

PROFESSOR ASKS

Why fix the random seed?

EXCELLENT ANSWER

Fixing the random seed ensures reproducibility. Every execution generates exactly the same noisy samples, allowing consistent experimental evaluation.

Random Number Generator

Your code creates

```
rng = np.random.default_rng(SEED)
```

Question. Why not use np.random.randn() Because default_rng() is NumPy's newer recommended random generator. It provides better randomness, better reproducibility, and cleaner design.

Loading Dataset

Your code uses pd.read_csv() Question. What happens? The complete master dataset is loaded into memory.

Wave Columns

Your code creates wave_cols = [f"v{i}" for i in range(N_PTS)] Question. Why? Because your dataset contains v0 v1 v2. v199 Instead of typing 200 names, Python creates them automatically. Excellent coding practice. Next Sample ID Your code computes next_id = df["sample_id"].max() + 1 Question. Why? Suppose the last sample has ID 815 The first noise sample should become 816 Otherwise, duplicate IDs would exist. Empty List noise_rows = [] **Purpose** — Store. every generated noise sample before saving.

Main Loop

Your code loops through

```
for circ in CIRCUITS:
```

Meaning **RC** — RLC. **SALLEN_KEY** — RC_LADDER. Each circuit gets its own noise class.

Selecting Normal Samples

One of the smartest lines in your code is normals = df[(df["circuit"] == circ) & (df["fault"] == "normal")] Question. Why only normal samples? Because measurement noise is naturally added to healthy circuits to create a distinct noise fault class. This avoids mixing multiple fault mechanisms during data generation.

PROFESSOR ASKS

Why didn't you add noise to every fault class?

EXCELLENT ANSWER

The objective was to introduce a separate noise fault class while keeping the original fault classes unchanged. This allows the classifier to learn noise as an independent operating condition without altering the characteristics of other faults. Number of Noise Samples Your code contains n_noise = len(normals) Question. Why? Because you generate exactly the same number of noise samples as normal samples. Example **Normal** — 51. **Noise** — 51. Balanced classes. Better ML training. Extracting Waveforms normal_waves = normals[wave_cols].values This gives a NumPy matrix containing only the waveform data. Metadata like sample_id fault circuit is ignored. Only voltages remain.

Why This Design Is Good

Instead of creating completely artificial waveforms, your script starts from physically correct normal waveforms and adds controlled noise. That is far more realistic than generating random voltage signals.

REVISION NOTES

Remember the first half of this script. **Load Dataset** — Choose Normal Samples. **Prepare Noise Generator** — Balance Classes. Extract Waveforms Everything after this will actually generate the Gaussian noise and append it to the dataset.

End of Chapter 24 (Part 1) Congratulations. You now understand why the noise generation script exists and the first half of its implementation. You can confidently explain: Why simulated datasets need measurement noise. What SNR represents. Why 25 dB was chosen. Why a random seed is fixed. Why only normal samples are used. How class balance is maintained. Author's Note The next continuation will cover Chapter 24 (Part 2): The mathematics of Gaussian noise. Signal power calculation. Noise power calculation. Why noise_std = sqrt(noise_power). rng.normal() explained line by line. How one noisy waveform is created. How the dataset expands from 815/816 samples to 1019 samples. Professor-level viva questions on noise modelling. Common misconceptions about SNR and Gaussian noise. This is one of the strongest theoretical sections of your project because it connects signal processing, statistics, and machine learning into a single implementation.

PART 2 — The Mathematics of Gaussian Noise Generation

Chapter 24 — Complete Python Source Code Explained Part 2 — day12b_generate_noise_samples.py (Based on your actual source code)

Before Starting

In Part 1,

we learned why the noise generator exists. Now comes the most interesting part. This is where your program actually creates realistic noisy signals. This chapter combines

Signal Processing,

Statistics, and Machine Learning. Where Are We? Our pipeline currently is

Master Dataset

Normal Waveforms

??? Now we generate Gaussian Noise.

Step 1 — Extracting One Waveform

Inside the loop, your code takes one waveform. Conceptually, it looks like wave = normal_waves[i] Question. What is wave? Answer A NumPy array containing 200 voltage samples. Example 0.00 0.13 0.24 0.41. 4.98 Nothing else. Only the voltage values.

Step 2 — Signal Power

One of the most important lines in the script. Your code computes

```
signal_power = np.mean(wave**2)
```

Question. What is Signal Power? Power is simply the average of the squared signal. Mathematically P = (1/N) Σ x^2 where x

Voltage Samples

Question. Why square? Because positive and negative voltages should both contribute positively to power. Exactly like RMS calculations.

PROFESSOR ASKS

Why is signal power calculated before adding noise?

EXCELLENT ANSWER

The signal power is required to determine the appropriate noise power for the desired Signal-to-Noise Ratio. Without knowing the signal power, we cannot generate correctly scaled noise.

Step 3 — Converting dB to Linear Scale

Your code uses the relationship between decibels and linear power. Conceptually, noise_power = signal_power / (10 ** (SNR_DB / 10)) Question. Why? Because 25 dB is not a power value. It is a logarithmic ratio.

Machine Learning

cannot use decibels directly. Need linear power. Example Suppose

Signal Power

100 **SNR** — 25 dB.

Noise Power

Much smaller than Exactly what we want.

PROFESSOR ASKS

Why divide by 10^(SNR/10)?

EXCELLENT ANSWER

Because SNR is specified in decibels, which is a logarithmic scale. The equation converts the desired SNR into a linear power ratio before calculating the required noise power.

Step 4 — Noise Standard Deviation

Your code computes

```
noise_std = np.sqrt(noise_power)
```

Question. Why square root? Remember statistics. For Gaussian distributions **Variance** — σ^2. **Standard deviation** — σ. Noise generators need the standard deviation, not the variance.

PROFESSOR ASKS

Why use the square root?

EXCELLENT ANSWER

The Gaussian noise generator requires the standard deviation. Since noise power corresponds to variance, the square root converts variance into standard deviation.

Step 5 — Generating Gaussian Noise

Now your code creates noise = rng.normal(0, noise_std, wave.shape) Question. What does this mean? **First parameter** — Mean. 0 Second parameter

Standard Deviation

noise_std **Third parameter** — Shape. 200 samples. Question. Why mean = 0? Because measurement noise should fluctuate equally above and below the signal. Example +0.01 -0.02 +0.03 -0.01 **Average** — Nearly zero. Exactly like real electronic noise.

PROFESSOR ASKS

Why is the Gaussian mean zero?

EXCELLENT ANSWER

Zero-mean Gaussian noise represents random measurement fluctuations without introducing a constant offset or bias into the waveform.

Step 6 — Creating the Noisy Signal

Your code adds the noise to the original waveform. Conceptually, noisy_wave = wave + noise Question. What happens? Example Original 5.00 Noise +0.03 Result 5.03 Another sample Original 4.98 Noise -0.02 Result 4.96 Every point changes slightly. The waveform still looks like the original, but now contains realistic noise.

Visual Idea

Original ———————— Noisy ~≈~≈~~≈~≈~≈~ Shape remains. Small random fluctuations appear.

Step 7 — Copying Metadata

Your code creates a copy of the original row. Conceptually, row = original.copy() Question. Why? Because everything except the waveform remains the same. **Circuit** — Same.

Parameter Value

Same Only the fault label and waveform change.

Step 8 — Changing the Fault Label

Your code changes fault = "noise" Question. Why? Because this waveform now belongs to a completely new class. The classifier must learn to distinguish noise from normal operation.

Step 9 — Replacing Waveform Values

Your code updates **v0** — new value. **v1** — new value. **v199** — new value. Question. Why? Because the original 200 voltages must be replaced by the noisy ones.

Step 10 — Adding the Sample

Now noise_rows.append(row) The newly generated noise waveform joins the growing list of new samples.

Final Step

After every circuit has been processed, your code combines both datasets. Conceptually,

```
final_df = pd.concat( [df, noise_df]
)
```

Question. What happens? Original dataset **Noise dataset** — One. larger dataset. Saving Finally, your code uses to_csv(.) to create the updated dataset. Now

Machine Learning

can train using both ideal and noisy signals.

Dataset Growth

Originally 815 (or 816) samples After noise generation 1019 samples Your dataset became larger and more realistic.

Why This Is Good Engineering

Most student projects train only on perfect data. Your project asks a much better question. "What happens when real measurement noise is present?" That makes the dataset closer to real-world conditions.

COMMON MISTAKES

MISTAKE: [X] Thinking noise destroys the waveform. No. At 25 dB, the signal still dominates.

MISTAKE: [X] Thinking Gaussian noise changes the fault. No. It changes the measurement, not the underlying circuit.

MISTAKE: [X] Thinking every run creates different research results. Not here. The fixed random seed ensures the same noisy dataset is produced every time.

REVISION NOTES

Remember the complete pipeline. **Normal Waveform** — Signal Power. **Noise Power** — Noise Standard Deviation. **Gaussian Noise** — Wave + Noise. **New Noise Sample** — Append Dataset. Save CSV Memory Trick Remember this sentence. "Measure the signal, compute the noise, generate Gaussian samples, add them to the waveform." That one sentence captures the entire algorithm.

KNOWLEDGE CHECK

Level 1

- What is SNR?
- Why did we choose 25 dB?
- Why is the random seed fixed?
- Why is Gaussian noise zero mean?
- Why do we calculate signal power first?

Level 2

- Explain how Gaussian noise is generated in your project.
- Why is noise power calculated from signal power?
- Why is standard deviation required instead of variance?
- Why are only normal waveforms used to create the noise class?
- How does adding noise improve the dataset?

Professor Level

- Explain the complete mathematical process of generating noisy waveforms.
- Why is Gaussian noise an appropriate model for measurement noise?
- How does SNR influence the amount of injected noise?
- Why is reproducibility important in machine learning experiments?
- How does this script make the overall project more suitable for real-world deployment?
- End of Chapter 24 (Part 2)
- Congratulations.
- You now understand the entire noise generation module, including the mathematics, implementation, and engineering motivation.
- This is one of the strongest parts of your project because it demonstrates that you considered realistic signal acquisition, not just ideal simulations.

Author's Note The next continuation begins Chapter 25 — day13_extract_features.py, one of the most important files in your project. We'll explain: Every one of the 17 extracted features. The mathematical formula behind each feature. Why each feature matters physically. Every function in the code, line by line. Why feature extraction is the heart of your machine learning pipeline. How the addition of DC Level transformed your classifier's performance. This chapter will be one of the longest and most important in the entire Project Bible.